Quantification des modèles de langage

Willy Rodríguez[1]

[1]Torus AI, 12 Avenue de l'Europe, 31520 Ramonville-Saint-Agne, France

Ce projet propose une étude mathématique et expérimentale de la quantification des modèles de langage. Les étudiants formaliseront la réduction de précision comme un problème d'approximation, puis analyseront la propagation de l'erreur dans un Transformer. Ils compareront les formats FP32, FP16, INT8 et INT4, et étudieront le compromis entre fidélité du modèle et mémoire. Le travail sera mené en Python et PyTorch sur un modèle de taille raisonnable, puis confronté à des méthodes récentes telles que GPTQ, SmoothQuant et AWQ.

Quantification | Modèles de langage | Analyse numérique | Deep Learning
Correspondence: willyrv@torus.ai

Introduction

Les modèles de langage de type Transformer comptent souvent plusieurs milliards de paramètres. Leur seule inférence exige donc beaucoup de mémoire et de calcul. La quantification réduit ce coût en représentant les poids, et parfois les activations, avec moins de bits : un flottant sur 32 ou 16 bits est approché par un entier sur 8 ou 4 bits (1).

Mathématiquement, on approche une matrice de poids W par une version quantifiée $Q(W)$, en contrôlant l'erreur $E = W - Q(W)$. Minimiser cette erreur sur les poids ne garantit pas que les sorties du réseau, ni la qualité du modèle, soient préservées : une perturbation peut être amplifiée par les couches suivantes (2).

L'enjeu est d'aborder cette question par l'analyse numérique. Les étudiants construiront d'abord leurs propres outils de quantification, avant de les comparer à des bibliothèques existantes, afin de relier erreur d'approximation, stabilité et compromis précision–mémoire.

Objectifs du projet

- Comprendre les formats FP32, FP16/BF16 et entiers, ainsi que la quantification uniforme, symétrique et asymétrique (*scale*, *zero-point*).
- Formaliser la quantification comme une approximation et comparer plusieurs mesures de l'erreur.
- Étudier la propagation d'une perturbation des poids dans une couche, puis dans un Transformer.
- Mesurer la sensibilité des couches et le compromis entre nombre de bits, mémoire et performance.
- Mettre en œuvre ces idées en Python et PyTorch, puis les situer par rapport à GPTQ, SmoothQuant et AWQ.

Méthodologie

1. **Étude mathématique.** Quantification uniforme sur b bits, choix du facteur d'échelle, effet du nombre de bits et des valeurs extrêmes. Le problème est posé comme la minimisation de $\|W - Q_s(W)\|$.
2. **Propagation de l'erreur.** Pour une couche $y = Wx$, l'écart de sortie vaut Ex. Des bornes en normes matricielles éclairent le conditionnement, puis l'analyse est étendue à une succession de couches.
3. **Expériences.** Implémentation simple en PyTorch, puis comparaison FP32, FP16, INT8 et INT4 : mémoire, perplexité, distribution des poids et sensibilité de chaque couche.
4. **Budget mémoire.** Choix d'une précision $b_l \in \{4, 8, 16\}$ par couche sous une contrainte de mémoire, par heuristiques ou méthode gloutonne, et lecture du compromis mémoire / performance.

Résultats attendus

Les étudiants remettront un rapport en anglais : formulation du problème, métriques d'erreur, propagation des perturbations, sensibilité des couches et compromis entre bits, mémoire et performance, avec une discussion critique des limites. Un dépôt Python documenté devra permettre de reproduire les expériences, accompagné de figures sur les poids, l'erreur par couche et la relation entre mémoire et performance.

Pour aller plus loin

Deux pistes peuvent prolonger le projet. La première compare la quantification après entraînement (*post-training quantization*) à celle prise en compte pendant l'apprentissage (*quantization-aware training*). La seconde confronte le travail réalisé à des méthodes conçues pour les grands modèles : GPTQ, qui utilise une information du second ordre (3), SmoothQuant, qui rééquilibre poids et activations (4), et AWQ, qui protège les poids jugés importants (5).

Bibliography

1. Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2704–2713, 2018.

2. Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for Transformers at scale. *Advances in Neural Information Processing Systems*, 2022.

3. Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *International Conference on Learning Representations*, 2023.

4. Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *International Conference on Machine Learning*, 2023.

5. Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *Proceedings of Machine Learning and Systems*, 2024.